GTOS Detailed Volume – Repository Evidence Atlas Repository Evidence Atlas

GTOS Enterprise Engineering Handbook - Final Edition 1.0

Source Chapters

- Repository Evidence Atlas Introduction
- Monorepo Topology and Root Inventory
- Frontend Application Registry
- Backend Gradle Module Registry
- Shared Package and Library Registry
- Service Runtime and Port Registry
- Database, Schema, and Migration Registry
- API and Route Evidence Registry
- Event, Topic, and Consumer Registry
- Workflow and Scheduled-Job Registry
- Deployment and Infrastructure Registry
- AI Module and Agent Registry
- Portal Route and Page Registry
- Test and Conformance Evidence Registry
- Observability and Operational Evidence Registry
- Legacy, Compatibility, and Retirement Registry
- Contradiction and Reconciliation Register
- Repository Evidence Classification Standard
- Component Ownership and RACI Matrix
- Repository Source Index and Traceability

Repository Evidence Atlas Introduction

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-00`
Subject	`RepositoryEvidenceAtlas`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Architecture Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **record observable repository reality at a verified commit and distinguish source, generated, legacy, planned, retired, and contradictory artifacts**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Architecture Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RepositoryEvidenceAtlas`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryEvidenceAtlasVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryEvidenceAtlasDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryEvidenceAtlasEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryEvidenceAtlasException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RepositoryEvidenceAtlas {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRepositoryEvidenceAtlas`
- `ValidateRepositoryEvidenceAtlas`
- `ApproveRepositoryEvidenceAtlas`
- `ActivateRepositoryEvidenceAtlas`
- `SuspendRepositoryEvidenceAtlas`
- `CorrectRepositoryEvidenceAtlas`
- `RetireRepositoryEvidenceAtlas`

Reference Events:

- `RepositoryEvidenceAtlasCreated`
- `RepositoryEvidenceAtlasValidated`
- `RepositoryEvidenceAtlasApproved`
- `RepositoryEvidenceAtlasActivated`
- `RepositoryEvidenceAtlasSuspended`
- `RepositoryEvidenceAtlasCorrected`
- `RepositoryEvidenceAtlasRetired`

Reference queries:

- `GetRepositoryEvidenceAtlas`
- `ListRepositoryEvidenceAtlasRecords`
- `GetRepositoryEvidenceAtlasTimeline`
- `GetRepositoryEvidenceAtlasEvidence`
- `GetRepositoryEvidenceAtlasDecisionPackage`
- `GetRepositoryEvidenceAtlasExceptions`
- `GetRepositoryEvidenceAtlasAuditTrail`

Reference API surface:

```
POST /api/v1/repositoryevidenceatlas/commands
GET  /api/v1/repositoryevidenceatlas/{id}
GET  /api/v1/repositoryevidenceatlas/{id}/timeline
GET  /api/v1/repositoryevidenceatlas/{id}/evidence
GET  /api/v1/repositoryevidenceatlas/{id}/decisions
POST /api/v1/repositoryevidenceatlas/{id}/exceptions
POST /api/v1/repositoryevidenceatlas/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`repositoryevidenceatlas_aggregate`	authoritative subject
`repositoryevidenceatlas_version`	immutable submitted, approved, issued, or signed versions
`repositoryevidenceatlas_relationship`	governed relationships
`repositoryevidenceatlas_decision`	binding Decisions and rationale
`repositoryevidenceatlas_evidence_link`	provenance and evidence relationships
`repositoryevidenceatlas_event_outbox`	transactional Event publication
`repositoryevidenceatlas_idempotency`	duplicate-write protection
`repositoryevidenceatlas_exception`	exception, claim, dispute, or hold
`repositoryevidenceatlas_correction`	correction and supersession lineage
`repositoryevidenceatlas_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Monorepo Topology and Root Inventory

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-01`
Subject	`RepositoryTopology`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Developer Platform
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **identify repository roots, applications, backend modules, packages, scripts, plans, infrastructure, generated artifacts, and archived evidence**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Developer Platform**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RepositoryTopology`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryTopologyVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryTopologyDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryTopologyEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositoryTopologyException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RepositoryTopology {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRepositoryTopology`
- `ValidateRepositoryTopology`
- `ApproveRepositoryTopology`
- `ActivateRepositoryTopology`
- `SuspendRepositoryTopology`
- `CorrectRepositoryTopology`
- `RetireRepositoryTopology`

Reference Events:

- `RepositoryTopologyCreated`
- `RepositoryTopologyValidated`
- `RepositoryTopologyApproved`
- `RepositoryTopologyActivated`
- `RepositoryTopologySuspended`
- `RepositoryTopologyCorrected`
- `RepositoryTopologyRetired`

Reference queries:

- `GetRepositoryTopology`
- `ListRepositoryTopologyRecords`
- `GetRepositoryTopologyTimeline`
- `GetRepositoryTopologyEvidence`
- `GetRepositoryTopologyDecisionPackage`
- `GetRepositoryTopologyExceptions`
- `GetRepositoryTopologyAuditTrail`

Reference API surface:

```
POST /api/v1/repositorytopology/commands
GET  /api/v1/repositorytopology/{id}
GET  /api/v1/repositorytopology/{id}/timeline
GET  /api/v1/repositorytopology/{id}/evidence
GET  /api/v1/repositorytopology/{id}/decisions
POST /api/v1/repositorytopology/{id}/exceptions
POST /api/v1/repositorytopology/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`repositorytopology_aggregate`	authoritative subject
`repositorytopology_version`	immutable submitted, approved, issued, or signed versions
`repositorytopology_relationship`	governed relationships
`repositorytopology_decision`	binding Decisions and rationale
`repositorytopology_evidence_link`	provenance and evidence relationships
`repositorytopology_event_outbox`	transactional Event publication
`repositorytopology_idempotency`	duplicate-write protection
`repositorytopology_exception`	exception, claim, dispute, or hold
`repositorytopology_correction`	correction and supersession lineage
`repositorytopology_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Frontend Application Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-02`
Subject	`FrontendApplicationRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Portal Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory every frontend application, runtime mode, route root, BFF dependency, session model, domain dependency, and lifecycle status**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Portal Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`FrontendApplicationRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`FrontendApplicationRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`FrontendApplicationRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`FrontendApplicationRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`FrontendApplicationRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
FrontendApplicationRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateFrontendApplicationRegistry`
- `ValidateFrontendApplicationRegistry`
- `ApproveFrontendApplicationRegistry`
- `ActivateFrontendApplicationRegistry`
- `SuspendFrontendApplicationRegistry`
- `CorrectFrontendApplicationRegistry`
- `RetireFrontendApplicationRegistry`

Reference Events:

- `FrontendApplicationRegistryCreated`
- `FrontendApplicationRegistryValidated`
- `FrontendApplicationRegistryApproved`
- `FrontendApplicationRegistryActivated`
- `FrontendApplicationRegistrySuspended`
- `FrontendApplicationRegistryCorrected`
- `FrontendApplicationRegistryRetired`

Reference queries:

- `GetFrontendApplicationRegistry`
- `ListFrontendApplicationRegistryRecords`
- `GetFrontendApplicationRegistryTimeline`
- `GetFrontendApplicationRegistryEvidence`
- `GetFrontendApplicationRegistryDecisionPackage`
- `GetFrontendApplicationRegistryExceptions`
- `GetFrontendApplicationRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/frontendapplicationregistry/commands
GET  /api/v1/frontendapplicationregistry/{id}
GET  /api/v1/frontendapplicationregistry/{id}/timeline
GET  /api/v1/frontendapplicationregistry/{id}/evidence
GET  /api/v1/frontendapplicationregistry/{id}/decisions
POST /api/v1/frontendapplicationregistry/{id}/exceptions
POST /api/v1/frontendapplicationregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`frontendapplicationregistry_aggregate`	authoritative subject
`frontendapplicationregistry_version`	immutable submitted, approved, issued, or signed versions
`frontendapplicationregistry_relationship`	governed relationships
`frontendapplicationregistry_decision`	binding Decisions and rationale
`frontendapplicationregistry_evidence_link`	provenance and evidence relationships
`frontendapplicationregistry_event_outbox`	transactional Event publication
`frontendapplicationregistry_idempotency`	duplicate-write protection
`frontendapplicationregistry_exception`	exception, claim, dispute, or hold
`frontendapplicationregistry_correction`	correction and supersession lineage
`frontendapplicationregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Backend Gradle Module Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-03`
Subject	`BackendModuleRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Backend Platform
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory every registered backend module, module type, path, owner, deployability, port, dependencies, tests, and lifecycle status**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Backend Platform**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`BackendModuleRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`BackendModuleRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`BackendModuleRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`BackendModuleRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`BackendModuleRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
BackendModuleRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateBackendModuleRegistry`
- `ValidateBackendModuleRegistry`
- `ApproveBackendModuleRegistry`
- `ActivateBackendModuleRegistry`
- `SuspendBackendModuleRegistry`
- `CorrectBackendModuleRegistry`
- `RetireBackendModuleRegistry`

Reference Events:

- `BackendModuleRegistryCreated`
- `BackendModuleRegistryValidated`
- `BackendModuleRegistryApproved`
- `BackendModuleRegistryActivated`
- `BackendModuleRegistrySuspended`
- `BackendModuleRegistryCorrected`
- `BackendModuleRegistryRetired`

Reference queries:

- `GetBackendModuleRegistry`
- `ListBackendModuleRegistryRecords`
- `GetBackendModuleRegistryTimeline`
- `GetBackendModuleRegistryEvidence`
- `GetBackendModuleRegistryDecisionPackage`
- `GetBackendModuleRegistryExceptions`
- `GetBackendModuleRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/backendmoduleregistry/commands
GET  /api/v1/backendmoduleregistry/{id}
GET  /api/v1/backendmoduleregistry/{id}/timeline
GET  /api/v1/backendmoduleregistry/{id}/evidence
GET  /api/v1/backendmoduleregistry/{id}/decisions
POST /api/v1/backendmoduleregistry/{id}/exceptions
POST /api/v1/backendmoduleregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`backendmoduleregistry_aggregate`	authoritative subject
`backendmoduleregistry_version`	immutable submitted, approved, issued, or signed versions
`backendmoduleregistry_relationship`	governed relationships
`backendmoduleregistry_decision`	binding Decisions and rationale
`backendmoduleregistry_evidence_link`	provenance and evidence relationships
`backendmoduleregistry_event_outbox`	transactional Event publication
`backendmoduleregistry_idempotency`	duplicate-write protection
`backendmoduleregistry_exception`	exception, claim, dispute, or hold
`backendmoduleregistry_correction`	correction and supersession lineage
`backendmoduleregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Shared Package and Library Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-04`
Subject	`SharedPackageRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Developer Platform
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory shared Java, TypeScript, UI, contract, client, schema, and build packages without treating shared code as shared Domain ownership**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Developer Platform**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`SharedPackageRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SharedPackageRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SharedPackageRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SharedPackageRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SharedPackageRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
SharedPackageRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateSharedPackageRegistry`
- `ValidateSharedPackageRegistry`
- `ApproveSharedPackageRegistry`
- `ActivateSharedPackageRegistry`
- `SuspendSharedPackageRegistry`
- `CorrectSharedPackageRegistry`
- `RetireSharedPackageRegistry`

Reference Events:

- `SharedPackageRegistryCreated`
- `SharedPackageRegistryValidated`
- `SharedPackageRegistryApproved`
- `SharedPackageRegistryActivated`
- `SharedPackageRegistrySuspended`
- `SharedPackageRegistryCorrected`
- `SharedPackageRegistryRetired`

Reference queries:

- `GetSharedPackageRegistry`
- `ListSharedPackageRegistryRecords`
- `GetSharedPackageRegistryTimeline`
- `GetSharedPackageRegistryEvidence`
- `GetSharedPackageRegistryDecisionPackage`
- `GetSharedPackageRegistryExceptions`
- `GetSharedPackageRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/sharedpackageregistry/commands
GET  /api/v1/sharedpackageregistry/{id}
GET  /api/v1/sharedpackageregistry/{id}/timeline
GET  /api/v1/sharedpackageregistry/{id}/evidence
GET  /api/v1/sharedpackageregistry/{id}/decisions
POST /api/v1/sharedpackageregistry/{id}/exceptions
POST /api/v1/sharedpackageregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`sharedpackageregistry_aggregate`	authoritative subject
`sharedpackageregistry_version`	immutable submitted, approved, issued, or signed versions
`sharedpackageregistry_relationship`	governed relationships
`sharedpackageregistry_decision`	binding Decisions and rationale
`sharedpackageregistry_evidence_link`	provenance and evidence relationships
`sharedpackageregistry_event_outbox`	transactional Event publication
`sharedpackageregistry_idempotency`	duplicate-write protection
`sharedpackageregistry_exception`	exception, claim, dispute, or hold
`sharedpackageregistry_correction`	correction and supersession lineage
`sharedpackageregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Service Runtime and Port Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-05`
Subject	`RuntimeServiceRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Platform Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **reconcile service names, paths, ports, health endpoints, deployment units, databases, schemas, and known consumers**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Platform Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RuntimeServiceRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RuntimeServiceRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RuntimeServiceRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RuntimeServiceRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RuntimeServiceRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RuntimeServiceRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRuntimeServiceRegistry`
- `ValidateRuntimeServiceRegistry`
- `ApproveRuntimeServiceRegistry`
- `ActivateRuntimeServiceRegistry`
- `SuspendRuntimeServiceRegistry`
- `CorrectRuntimeServiceRegistry`
- `RetireRuntimeServiceRegistry`

Reference Events:

- `RuntimeServiceRegistryCreated`
- `RuntimeServiceRegistryValidated`
- `RuntimeServiceRegistryApproved`
- `RuntimeServiceRegistryActivated`
- `RuntimeServiceRegistrySuspended`
- `RuntimeServiceRegistryCorrected`
- `RuntimeServiceRegistryRetired`

Reference queries:

- `GetRuntimeServiceRegistry`
- `ListRuntimeServiceRegistryRecords`
- `GetRuntimeServiceRegistryTimeline`
- `GetRuntimeServiceRegistryEvidence`
- `GetRuntimeServiceRegistryDecisionPackage`
- `GetRuntimeServiceRegistryExceptions`
- `GetRuntimeServiceRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/runtimeserviceregistry/commands
GET  /api/v1/runtimeserviceregistry/{id}
GET  /api/v1/runtimeserviceregistry/{id}/timeline
GET  /api/v1/runtimeserviceregistry/{id}/evidence
GET  /api/v1/runtimeserviceregistry/{id}/decisions
POST /api/v1/runtimeserviceregistry/{id}/exceptions
POST /api/v1/runtimeserviceregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`runtimeserviceregistry_aggregate`	authoritative subject
`runtimeserviceregistry_version`	immutable submitted, approved, issued, or signed versions
`runtimeserviceregistry_relationship`	governed relationships
`runtimeserviceregistry_decision`	binding Decisions and rationale
`runtimeserviceregistry_evidence_link`	provenance and evidence relationships
`runtimeserviceregistry_event_outbox`	transactional Event publication
`runtimeserviceregistry_idempotency`	duplicate-write protection
`runtimeserviceregistry_exception`	exception, claim, dispute, or hold
`runtimeserviceregistry_correction`	correction and supersession lineage
`runtimeserviceregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Database, Schema, and Migration Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-06`
Subject	`DatabaseRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Data Architecture Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **reconcile database names, schema owners, migrations, table families, users, RLS, backups, consumers, and contradictory ownership claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Data Architecture Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`DatabaseRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DatabaseRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DatabaseRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DatabaseRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DatabaseRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
DatabaseRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateDatabaseRegistry`
- `ValidateDatabaseRegistry`
- `ApproveDatabaseRegistry`
- `ActivateDatabaseRegistry`
- `SuspendDatabaseRegistry`
- `CorrectDatabaseRegistry`
- `RetireDatabaseRegistry`

Reference Events:

- `DatabaseRegistryCreated`
- `DatabaseRegistryValidated`
- `DatabaseRegistryApproved`
- `DatabaseRegistryActivated`
- `DatabaseRegistrySuspended`
- `DatabaseRegistryCorrected`
- `DatabaseRegistryRetired`

Reference queries:

- `GetDatabaseRegistry`
- `ListDatabaseRegistryRecords`
- `GetDatabaseRegistryTimeline`
- `GetDatabaseRegistryEvidence`
- `GetDatabaseRegistryDecisionPackage`
- `GetDatabaseRegistryExceptions`
- `GetDatabaseRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/databaseregistry/commands
GET  /api/v1/databaseregistry/{id}
GET  /api/v1/databaseregistry/{id}/timeline
GET  /api/v1/databaseregistry/{id}/evidence
GET  /api/v1/databaseregistry/{id}/decisions
POST /api/v1/databaseregistry/{id}/exceptions
POST /api/v1/databaseregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`databaseregistry_aggregate`	authoritative subject
`databaseregistry_version`	immutable submitted, approved, issued, or signed versions
`databaseregistry_relationship`	governed relationships
`databaseregistry_decision`	binding Decisions and rationale
`databaseregistry_evidence_link`	provenance and evidence relationships
`databaseregistry_event_outbox`	transactional Event publication
`databaseregistry_idempotency`	duplicate-write protection
`databaseregistry_exception`	exception, claim, dispute, or hold
`databaseregistry_correction`	correction and supersession lineage
`databaseregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

API and Route Evidence Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-07`
Subject	`RouteRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Contract Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory backend routes, BFF routes, frontend client contracts, OpenAPI evidence, authentication, authorization, versioning, and missing implementations**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Contract Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RouteRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RouteRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RouteRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RouteRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RouteRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RouteRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRouteRegistry`
- `ValidateRouteRegistry`
- `ApproveRouteRegistry`
- `ActivateRouteRegistry`
- `SuspendRouteRegistry`
- `CorrectRouteRegistry`
- `RetireRouteRegistry`

Reference Events:

- `RouteRegistryCreated`
- `RouteRegistryValidated`
- `RouteRegistryApproved`
- `RouteRegistryActivated`
- `RouteRegistrySuspended`
- `RouteRegistryCorrected`
- `RouteRegistryRetired`

Reference queries:

- `GetRouteRegistry`
- `ListRouteRegistryRecords`
- `GetRouteRegistryTimeline`
- `GetRouteRegistryEvidence`
- `GetRouteRegistryDecisionPackage`
- `GetRouteRegistryExceptions`
- `GetRouteRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/routeregistry/commands
GET  /api/v1/routeregistry/{id}
GET  /api/v1/routeregistry/{id}/timeline
GET  /api/v1/routeregistry/{id}/evidence
GET  /api/v1/routeregistry/{id}/decisions
POST /api/v1/routeregistry/{id}/exceptions
POST /api/v1/routeregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`routeregistry_aggregate`	authoritative subject
`routeregistry_version`	immutable submitted, approved, issued, or signed versions
`routeregistry_relationship`	governed relationships
`routeregistry_decision`	binding Decisions and rationale
`routeregistry_evidence_link`	provenance and evidence relationships
`routeregistry_event_outbox`	transactional Event publication
`routeregistry_idempotency`	duplicate-write protection
`routeregistry_exception`	exception, claim, dispute, or hold
`routeregistry_correction`	correction and supersession lineage
`routeregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Event, Topic, and Consumer Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-08`
Subject	`EventRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Event Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory Event names, producers, topics, schemas, outboxes, consumers, replay, DLQ, ordering, and ownership**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Event Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`EventRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EventRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EventRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EventRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EventRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
EventRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateEventRegistry`
- `ValidateEventRegistry`
- `ApproveEventRegistry`
- `ActivateEventRegistry`
- `SuspendEventRegistry`
- `CorrectEventRegistry`
- `RetireEventRegistry`

Reference Events:

- `EventRegistryCreated`
- `EventRegistryValidated`
- `EventRegistryApproved`
- `EventRegistryActivated`
- `EventRegistrySuspended`
- `EventRegistryCorrected`
- `EventRegistryRetired`

Reference queries:

- `GetEventRegistry`
- `ListEventRegistryRecords`
- `GetEventRegistryTimeline`
- `GetEventRegistryEvidence`
- `GetEventRegistryDecisionPackage`
- `GetEventRegistryExceptions`
- `GetEventRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/eventregistry/commands
GET  /api/v1/eventregistry/{id}
GET  /api/v1/eventregistry/{id}/timeline
GET  /api/v1/eventregistry/{id}/evidence
GET  /api/v1/eventregistry/{id}/decisions
POST /api/v1/eventregistry/{id}/exceptions
POST /api/v1/eventregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`eventregistry_aggregate`	authoritative subject
`eventregistry_version`	immutable submitted, approved, issued, or signed versions
`eventregistry_relationship`	governed relationships
`eventregistry_decision`	binding Decisions and rationale
`eventregistry_evidence_link`	provenance and evidence relationships
`eventregistry_event_outbox`	transactional Event publication
`eventregistry_idempotency`	duplicate-write protection
`eventregistry_exception`	exception, claim, dispute, or hold
`eventregistry_correction`	correction and supersession lineage
`eventregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Workflow and Scheduled-Job Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-09`
Subject	`WorkflowRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Workflow Platform
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory durable workflows, approvals, timers, retries, compensation, scheduled jobs, batch jobs, and orphan risks**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Workflow Platform**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`WorkflowRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WorkflowRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WorkflowRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WorkflowRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WorkflowRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
WorkflowRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateWorkflowRegistry`
- `ValidateWorkflowRegistry`
- `ApproveWorkflowRegistry`
- `ActivateWorkflowRegistry`
- `SuspendWorkflowRegistry`
- `CorrectWorkflowRegistry`
- `RetireWorkflowRegistry`

Reference Events:

- `WorkflowRegistryCreated`
- `WorkflowRegistryValidated`
- `WorkflowRegistryApproved`
- `WorkflowRegistryActivated`
- `WorkflowRegistrySuspended`
- `WorkflowRegistryCorrected`
- `WorkflowRegistryRetired`

Reference queries:

- `GetWorkflowRegistry`
- `ListWorkflowRegistryRecords`
- `GetWorkflowRegistryTimeline`
- `GetWorkflowRegistryEvidence`
- `GetWorkflowRegistryDecisionPackage`
- `GetWorkflowRegistryExceptions`
- `GetWorkflowRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/workflowregistry/commands
GET  /api/v1/workflowregistry/{id}
GET  /api/v1/workflowregistry/{id}/timeline
GET  /api/v1/workflowregistry/{id}/evidence
GET  /api/v1/workflowregistry/{id}/decisions
POST /api/v1/workflowregistry/{id}/exceptions
POST /api/v1/workflowregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`workflowregistry_aggregate`	authoritative subject
`workflowregistry_version`	immutable submitted, approved, issued, or signed versions
`workflowregistry_relationship`	governed relationships
`workflowregistry_decision`	binding Decisions and rationale
`workflowregistry_evidence_link`	provenance and evidence relationships
`workflowregistry_event_outbox`	transactional Event publication
`workflowregistry_idempotency`	duplicate-write protection
`workflowregistry_exception`	exception, claim, dispute, or hold
`workflowregistry_correction`	correction and supersession lineage
`workflowregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Deployment and Infrastructure Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-10`
Subject	`DeploymentRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Infrastructure Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory Compose, Kubernetes, Kustomize, Helm, Istio, Terraform, storage, ingress, secrets, observability, and environment overlays**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Infrastructure Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`DeploymentRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DeploymentRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DeploymentRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DeploymentRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`DeploymentRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
DeploymentRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateDeploymentRegistry`
- `ValidateDeploymentRegistry`
- `ApproveDeploymentRegistry`
- `ActivateDeploymentRegistry`
- `SuspendDeploymentRegistry`
- `CorrectDeploymentRegistry`
- `RetireDeploymentRegistry`

Reference Events:

- `DeploymentRegistryCreated`
- `DeploymentRegistryValidated`
- `DeploymentRegistryApproved`
- `DeploymentRegistryActivated`
- `DeploymentRegistrySuspended`
- `DeploymentRegistryCorrected`
- `DeploymentRegistryRetired`

Reference queries:

- `GetDeploymentRegistry`
- `ListDeploymentRegistryRecords`
- `GetDeploymentRegistryTimeline`
- `GetDeploymentRegistryEvidence`
- `GetDeploymentRegistryDecisionPackage`
- `GetDeploymentRegistryExceptions`
- `GetDeploymentRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/deploymentregistry/commands
GET  /api/v1/deploymentregistry/{id}
GET  /api/v1/deploymentregistry/{id}/timeline
GET  /api/v1/deploymentregistry/{id}/evidence
GET  /api/v1/deploymentregistry/{id}/decisions
POST /api/v1/deploymentregistry/{id}/exceptions
POST /api/v1/deploymentregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`deploymentregistry_aggregate`	authoritative subject
`deploymentregistry_version`	immutable submitted, approved, issued, or signed versions
`deploymentregistry_relationship`	governed relationships
`deploymentregistry_decision`	binding Decisions and rationale
`deploymentregistry_evidence_link`	provenance and evidence relationships
`deploymentregistry_event_outbox`	transactional Event publication
`deploymentregistry_idempotency`	duplicate-write protection
`deploymentregistry_exception`	exception, claim, dispute, or hold
`deploymentregistry_correction`	correction and supersession lineage
`deploymentregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

AI Module and Agent Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-11`
Subject	`AIModuleRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	AI Architecture Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory AI runtime, orchestrator, intelligence services, models, providers, Agents, Tools, memory, RAG, evaluation, and duplicate ownership**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **AI Architecture Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`AIModuleRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AIModuleRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AIModuleRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AIModuleRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AIModuleRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
AIModuleRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateAIModuleRegistry`
- `ValidateAIModuleRegistry`
- `ApproveAIModuleRegistry`
- `ActivateAIModuleRegistry`
- `SuspendAIModuleRegistry`
- `CorrectAIModuleRegistry`
- `RetireAIModuleRegistry`

Reference Events:

- `AIModuleRegistryCreated`
- `AIModuleRegistryValidated`
- `AIModuleRegistryApproved`
- `AIModuleRegistryActivated`
- `AIModuleRegistrySuspended`
- `AIModuleRegistryCorrected`
- `AIModuleRegistryRetired`

Reference queries:

- `GetAIModuleRegistry`
- `ListAIModuleRegistryRecords`
- `GetAIModuleRegistryTimeline`
- `GetAIModuleRegistryEvidence`
- `GetAIModuleRegistryDecisionPackage`
- `GetAIModuleRegistryExceptions`
- `GetAIModuleRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/aimoduleregistry/commands
GET  /api/v1/aimoduleregistry/{id}
GET  /api/v1/aimoduleregistry/{id}/timeline
GET  /api/v1/aimoduleregistry/{id}/evidence
GET  /api/v1/aimoduleregistry/{id}/decisions
POST /api/v1/aimoduleregistry/{id}/exceptions
POST /api/v1/aimoduleregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`aimoduleregistry_aggregate`	authoritative subject
`aimoduleregistry_version`	immutable submitted, approved, issued, or signed versions
`aimoduleregistry_relationship`	governed relationships
`aimoduleregistry_decision`	binding Decisions and rationale
`aimoduleregistry_evidence_link`	provenance and evidence relationships
`aimoduleregistry_event_outbox`	transactional Event publication
`aimoduleregistry_idempotency`	duplicate-write protection
`aimoduleregistry_exception`	exception, claim, dispute, or hold
`aimoduleregistry_correction`	correction and supersession lineage
`aimoduleregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Portal Route and Page Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-12`
Subject	`PortalRouteRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Experience Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory page routes, layouts, components, client APIs, role gates, loading/error states, analytics, and test journeys**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Experience Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;

- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`PortalRouteRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PortalRouteRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PortalRouteRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PortalRouteRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PortalRouteRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
PortalRouteRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
```

```
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreatePortalRouteRegistry`
- `ValidatePortalRouteRegistry`
- `ApprovePortalRouteRegistry`
- `ActivatePortalRouteRegistry`
- `SuspendPortalRouteRegistry`
- `CorrectPortalRouteRegistry`
- `RetirePortalRouteRegistry`

Reference Events:

- `PortalRouteRegistryCreated`
- `PortalRouteRegistryValidated`
- `PortalRouteRegistryApproved`
- `PortalRouteRegistryActivated`
- `PortalRouteRegistrySuspended`
- `PortalRouteRegistryCorrected`
- `PortalRouteRegistryRetired`

Reference queries:

- `GetPortalRouteRegistry`
- `ListPortalRouteRegistryRecords`
- `GetPortalRouteRegistryTimeline`
- `GetPortalRouteRegistryEvidence`
- `GetPortalRouteRegistryDecisionPackage`
- `GetPortalRouteRegistryExceptions`
- `GetPortalRouteRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/portalrouteregistry/commands
GET  /api/v1/portalrouteregistry/{id}
GET  /api/v1/portalrouteregistry/{id}/timeline
GET  /api/v1/portalrouteregistry/{id}/evidence
GET  /api/v1/portalrouteregistry/{id}/decisions
POST /api/v1/portalrouteregistry/{id}/exceptions
POST /api/v1/portalrouteregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
```

```
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`portalrouteregistry_aggregate`	authoritative subject
`portalrouteregistry_version`	immutable submitted, approved, issued, or signed versions
`portalrouteregistry_relationship`	governed relationships
`portalrouteregistry_decision`	binding Decisions and rationale
`portalrouteregistry_evidence_link`	provenance and evidence relationships
`portalrouteregistry_event_outbox`	transactional Event publication
`portalrouteregistry_idempotency`	duplicate-write protection
`portalrouteregistry_exception`	exception, claim, dispute, or hold
`portalrouteregistry_correction`	correction and supersession lineage
`portalrouteregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Test and Conformance Evidence Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-13`
Subject	`TestEvidenceRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Quality Engineering
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory unit, integration, contract, migration, architecture, security, E2E, performance, recovery, and AI evaluation evidence**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Quality Engineering**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`TestEvidenceRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`TestEvidenceRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`TestEvidenceRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`TestEvidenceRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`TestEvidenceRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
TestEvidenceRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateTestEvidenceRegistry`
- `ValidateTestEvidenceRegistry`
- `ApproveTestEvidenceRegistry`
- `ActivateTestEvidenceRegistry`
- `SuspendTestEvidenceRegistry`
- `CorrectTestEvidenceRegistry`
- `RetireTestEvidenceRegistry`

Reference Events:

- `TestEvidenceRegistryCreated`
- `TestEvidenceRegistryValidated`
- `TestEvidenceRegistryApproved`
- `TestEvidenceRegistryActivated`
- `TestEvidenceRegistrySuspended`
- `TestEvidenceRegistryCorrected`
- `TestEvidenceRegistryRetired`

Reference queries:

- `GetTestEvidenceRegistry`
- `ListTestEvidenceRegistryRecords`
- `GetTestEvidenceRegistryTimeline`
- `GetTestEvidenceRegistryEvidence`
- `GetTestEvidenceRegistryDecisionPackage`
- `GetTestEvidenceRegistryExceptions`
- `GetTestEvidenceRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/testevidenceregistry/commands
GET  /api/v1/testevidenceregistry/{id}
GET  /api/v1/testevidenceregistry/{id}/timeline
GET  /api/v1/testevidenceregistry/{id}/evidence
GET  /api/v1/testevidenceregistry/{id}/decisions
POST /api/v1/testevidenceregistry/{id}/exceptions
POST /api/v1/testevidenceregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`testevidenceregistry_aggregate`	authoritative subject
`testevidenceregistry_version`	immutable submitted, approved, issued, or signed versions
`testevidenceregistry_relationship`	governed relationships
`testevidenceregistry_decision`	binding Decisions and rationale
`testevidenceregistry_evidence_link`	provenance and evidence relationships
`testevidenceregistry_event_outbox`	transactional Event publication
`testevidenceregistry_idempotency`	duplicate-write protection
`testevidenceregistry_exception`	exception, claim, dispute, or hold
`testevidenceregistry_correction`	correction and supersession lineage
`testevidenceregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Observability and Operational Evidence Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-14`
Subject	`OperationalEvidenceRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	SRE Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **inventory logs, metrics, traces, dashboards, alerts, SLOs, runbooks, ownership, incidents, and recovery evidence**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **SRE Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`OperationalEvidenceRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`OperationalEvidenceRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`OperationalEvidenceRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`OperationalEvidenceRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`OperationalEvidenceRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
OperationalEvidenceRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateOperationalEvidenceRegistry`
- `ValidateOperationalEvidenceRegistry`
- `ApproveOperationalEvidenceRegistry`
- `ActivateOperationalEvidenceRegistry`
- `SuspendOperationalEvidenceRegistry`
- `CorrectOperationalEvidenceRegistry`
- `RetireOperationalEvidenceRegistry`

Reference Events:

- `OperationalEvidenceRegistryCreated`
- `OperationalEvidenceRegistryValidated`
- `OperationalEvidenceRegistryApproved`
- `OperationalEvidenceRegistryActivated`
- `OperationalEvidenceRegistrySuspended`
- `OperationalEvidenceRegistryCorrected`
- `OperationalEvidenceRegistryRetired`

Reference queries:

- `GetOperationalEvidenceRegistry`
- `ListOperationalEvidenceRegistryRecords`
- `GetOperationalEvidenceRegistryTimeline`
- `GetOperationalEvidenceRegistryEvidence`
- `GetOperationalEvidenceRegistryDecisionPackage`
- `GetOperationalEvidenceRegistryExceptions`
- `GetOperationalEvidenceRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/operationalevidenceregistry/commands
GET  /api/v1/operationalevidenceregistry/{id}
GET  /api/v1/operationalevidenceregistry/{id}/timeline
GET  /api/v1/operationalevidenceregistry/{id}/evidence
GET  /api/v1/operationalevidenceregistry/{id}/decisions
POST /api/v1/operationalevidenceregistry/{id}/exceptions
POST /api/v1/operationalevidenceregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`operationalevidenceregistry_aggregate`	authoritative subject
`operationalevidenceregistry_version`	immutable submitted, approved, issued, or signed versions
`operationalevidenceregistry_relationship`	governed relationships
`operationalevidenceregistry_decision`	binding Decisions and rationale
`operationalevidenceregistry_evidence_link`	provenance and evidence relationships
`operationalevidenceregistry_event_outbox`	transactional Event publication
`operationalevidenceregistry_idempotency`	duplicate-write protection
`operationalevidenceregistry_exception`	exception, claim, dispute, or hold
`operationalevidenceregistry_correction`	correction and supersession lineage
`operationalevidenceregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Legacy, Compatibility, and Retirement Registry

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-15`
Subject	`LegacyRegistry`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Evolution Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **identify compatibility modules, deleted paths, duplicate implementations, adapters, deprecated APIs, retired tables, and decommissioning evidence**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Evolution Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`LegacyRegistry`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`LegacyRegistryVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`LegacyRegistryDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`LegacyRegistryEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`LegacyRegistryException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
LegacyRegistry {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateLegacyRegistry`
- `ValidateLegacyRegistry`
- `ApproveLegacyRegistry`
- `ActivateLegacyRegistry`
- `SuspendLegacyRegistry`
- `CorrectLegacyRegistry`
- `RetireLegacyRegistry`

Reference Events:

- `LegacyRegistryCreated`
- `LegacyRegistryValidated`
- `LegacyRegistryApproved`
- `LegacyRegistryActivated`
- `LegacyRegistrySuspended`
- `LegacyRegistryCorrected`
- `LegacyRegistryRetired`

Reference queries:

- `GetLegacyRegistry`
- `ListLegacyRegistryRecords`
- `GetLegacyRegistryTimeline`
- `GetLegacyRegistryEvidence`
- `GetLegacyRegistryDecisionPackage`
- `GetLegacyRegistryExceptions`
- `GetLegacyRegistryAuditTrail`

Reference API surface:

```
POST /api/v1/legacyregistry/commands
GET  /api/v1/legacyregistry/{id}
GET  /api/v1/legacyregistry/{id}/timeline
GET  /api/v1/legacyregistry/{id}/evidence
GET  /api/v1/legacyregistry/{id}/decisions
POST /api/v1/legacyregistry/{id}/exceptions
POST /api/v1/legacyregistry/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`legacyregistry_aggregate`	authoritative subject
`legacyregistry_version`	immutable submitted, approved, issued, or signed versions
`legacyregistry_relationship`	governed relationships
`legacyregistry_decision`	binding Decisions and rationale
`legacyregistry_evidence_link`	provenance and evidence relationships
`legacyregistry_event_outbox`	transactional Event publication
`legacyregistry_idempotency`	duplicate-write protection
`legacyregistry_exception`	exception, claim, dispute, or hold
`legacyregistry_correction`	correction and supersession lineage
`legacyregistry_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Contradiction and Reconciliation Register

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-16`
Subject	`ContradictionRegister`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Architecture Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **record conflicting counts, paths, names, ports, databases, owners, statuses, and claims without silently choosing a winner**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Architecture Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ContradictionRegister`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ContradictionRegisterVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ContradictionRegisterDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ContradictionRegisterEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ContradictionRegisterException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ContradictionRegister {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateContradictionRegister`
- `ValidateContradictionRegister`
- `ApproveContradictionRegister`
- `ActivateContradictionRegister`
- `SuspendContradictionRegister`
- `CorrectContradictionRegister`
- `RetireContradictionRegister`

Reference Events:

- `ContradictionRegisterCreated`
- `ContradictionRegisterValidated`
- `ContradictionRegisterApproved`
- `ContradictionRegisterActivated`
- `ContradictionRegisterSuspended`
- `ContradictionRegisterCorrected`
- `ContradictionRegisterRetired`

Reference queries:

- `GetContradictionRegister`
- `ListContradictionRegisterRecords`
- `GetContradictionRegisterTimeline`
- `GetContradictionRegisterEvidence`
- `GetContradictionRegisterDecisionPackage`
- `GetContradictionRegisterExceptions`
- `GetContradictionRegisterAuditTrail`

Reference API surface:

```
POST /api/v1/contradictionregister/commands
GET  /api/v1/contradictionregister/{id}
GET  /api/v1/contradictionregister/{id}/timeline
GET  /api/v1/contradictionregister/{id}/evidence
GET  /api/v1/contradictionregister/{id}/decisions
POST /api/v1/contradictionregister/{id}/exceptions
POST /api/v1/contradictionregister/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`contradictionregister_aggregate`	authoritative subject
`contradictionregister_version`	immutable submitted, approved, issued, or signed versions
`contradictionregister_relationship`	governed relationships
`contradictionregister_decision`	binding Decisions and rationale
`contradictionregister_evidence_link`	provenance and evidence relationships
`contradictionregister_event_outbox`	transactional Event publication
`contradictionregister_idempotency`	duplicate-write protection
`contradictionregister_exception`	exception, claim, dispute, or hold
`contradictionregister_correction`	correction and supersession lineage
`contradictionregister_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Repository Evidence Classification Standard

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-17`
Subject	`EvidenceClassification`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Architecture Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the evidence hierarchy from executable source and runtime evidence through documentation-only and planned claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Architecture Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`EvidenceClassification`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EvidenceClassificationVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EvidenceClassificationDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EvidenceClassificationEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`EvidenceClassificationException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
EvidenceClassification {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateEvidenceClassification`
- `ValidateEvidenceClassification`
- `ApproveEvidenceClassification`
- `ActivateEvidenceClassification`
- `SuspendEvidenceClassification`
- `CorrectEvidenceClassification`
- `RetireEvidenceClassification`

Reference Events:

- `EvidenceClassificationCreated`
- `EvidenceClassificationValidated`
- `EvidenceClassificationApproved`
- `EvidenceClassificationActivated`
- `EvidenceClassificationSuspended`
- `EvidenceClassificationCorrected`
- `EvidenceClassificationRetired`

Reference queries:

- `GetEvidenceClassification`
- `ListEvidenceClassificationRecords`
- `GetEvidenceClassificationTimeline`
- `GetEvidenceClassificationEvidence`
- `GetEvidenceClassificationDecisionPackage`
- `GetEvidenceClassificationExceptions`
- `GetEvidenceClassificationAuditTrail`

Reference API surface:

```
POST /api/v1/evidenceclassification/commands
GET  /api/v1/evidenceclassification/{id}
GET  /api/v1/evidenceclassification/{id}/timeline
GET  /api/v1/evidenceclassification/{id}/evidence
GET  /api/v1/evidenceclassification/{id}/decisions
POST /api/v1/evidenceclassification/{id}/exceptions
POST /api/v1/evidenceclassification/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`evidenceclassification_aggregate`	authoritative subject
`evidenceclassification_version`	immutable submitted, approved, issued, or signed versions
`evidenceclassification_relationship`	governed relationships
`evidenceclassification_decision`	binding Decisions and rationale
`evidenceclassification_evidence_link`	provenance and evidence relationships
`evidenceclassification_event_outbox`	transactional Event publication
`evidenceclassification_idempotency`	duplicate-write protection
`evidenceclassification_exception`	exception, claim, dispute, or hold
`evidenceclassification_correction`	correction and supersession lineage
`evidenceclassification_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Component Ownership and RACI Matrix

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-18`
Subject	`ComponentOwnershipMatrix`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Enterprise Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **assign accountable Domain, technical owner, operator, approver, consumers, escalation, and exception authority for every component**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Enterprise Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ComponentOwnershipMatrix`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ComponentOwnershipMatrixVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ComponentOwnershipMatrixDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ComponentOwnershipMatrixEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ComponentOwnershipMatrixException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ComponentOwnershipMatrix {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateComponentOwnershipMatrix`
- `ValidateComponentOwnershipMatrix`
- `ApproveComponentOwnershipMatrix`
- `ActivateComponentOwnershipMatrix`
- `SuspendComponentOwnershipMatrix`
- `CorrectComponentOwnershipMatrix`
- `RetireComponentOwnershipMatrix`

Reference Events:

- `ComponentOwnershipMatrixCreated`
- `ComponentOwnershipMatrixValidated`
- `ComponentOwnershipMatrixApproved`
- `ComponentOwnershipMatrixActivated`
- `ComponentOwnershipMatrixSuspended`
- `ComponentOwnershipMatrixCorrected`
- `ComponentOwnershipMatrixRetired`

Reference queries:

- `GetComponentOwnershipMatrix`
- `ListComponentOwnershipMatrixRecords`
- `GetComponentOwnershipMatrixTimeline`
- `GetComponentOwnershipMatrixEvidence`
- `GetComponentOwnershipMatrixDecisionPackage`
- `GetComponentOwnershipMatrixExceptions`
- `GetComponentOwnershipMatrixAuditTrail`

Reference API surface:

```
POST /api/v1/componentownershipmatrix/commands
GET  /api/v1/componentownershipmatrix/{id}
GET  /api/v1/componentownershipmatrix/{id}/timeline
GET  /api/v1/componentownershipmatrix/{id}/evidence
GET  /api/v1/componentownershipmatrix/{id}/decisions
POST /api/v1/componentownershipmatrix/{id}/exceptions
POST /api/v1/componentownershipmatrix/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`componentownershipmatrix_aggregate`	authoritative subject
`componentownershipmatrix_version`	immutable submitted, approved, issued, or signed versions
`componentownershipmatrix_relationship`	governed relationships
`componentownershipmatrix_decision`	binding Decisions and rationale
`componentownershipmatrix_evidence_link`	provenance and evidence relationships
`componentownershipmatrix_event_outbox`	transactional Event publication
`componentownershipmatrix_idempotency`	duplicate-write protection
`componentownershipmatrix_exception`	exception, claim, dispute, or hold
`componentownershipmatrix_correction`	correction and supersession lineage
`componentownershipmatrix_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Repository Source Index and Traceability

Metadata	Value
Volume	`-2`
Chapter ID	`V-02-19`
Subject	`RepositorySourceIndex`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Documentation Architecture
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **link Canons, Articles, capabilities, lifecycle stages, Domains, services, tables, APIs, Events, workflows, Portals, AI Tools, tests, and repository evidence**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Documentation Architecture**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RepositorySourceIndex`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositorySourceIndexVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositorySourceIndexDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositorySourceIndexEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RepositorySourceIndexException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RepositorySourceIndex {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
PROPOSED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
VALIDATION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
APPROVAL_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
ACTIVE	Distinct governed condition with entry, exit, timeout, correction and authority semantics
SUSPENDED	Distinct governed condition with entry, exit, timeout, correction and authority semantics
CORRECTION_PENDING	Distinct governed condition with entry, exit, timeout, correction and authority semantics
RETIRED	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRepositorySourceIndex`
- `ValidateRepositorySourceIndex`
- `ApproveRepositorySourceIndex`
- `ActivateRepositorySourceIndex`
- `SuspendRepositorySourceIndex`
- `CorrectRepositorySourceIndex`
- `RetireRepositorySourceIndex`

Reference Events:

- `RepositorySourceIndexCreated`
- `RepositorySourceIndexValidated`
- `RepositorySourceIndexApproved`
- `RepositorySourceIndexActivated`
- `RepositorySourceIndexSuspended`
- `RepositorySourceIndexCorrected`
- `RepositorySourceIndexRetired`

Reference queries:

- `GetRepositorySourceIndex`
- `ListRepositorySourceIndexRecords`
- `GetRepositorySourceIndexTimeline`
- `GetRepositorySourceIndexEvidence`
- `GetRepositorySourceIndexDecisionPackage`
- `GetRepositorySourceIndexExceptions`
- `GetRepositorySourceIndexAuditTrail`

Reference API surface:

```
POST /api/v1/repositorysourceindex/commands
GET  /api/v1/repositorysourceindex/{id}
GET  /api/v1/repositorysourceindex/{id}/timeline
GET  /api/v1/repositorysourceindex/{id}/evidence
GET  /api/v1/repositorysourceindex/{id}/decisions
POST /api/v1/repositorysourceindex/{id}/exceptions
POST /api/v1/repositorysourceindex/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`repositorysourceindex_aggregate`	authoritative subject
`repositorysourceindex_version`	immutable submitted, approved, issued, or signed versions
`repositorysourceindex_relationship`	governed relationships
`repositorysourceindex_decision`	binding Decisions and rationale
`repositorysourceindex_evidence_link`	provenance and evidence relationships
`repositorysourceindex_event_outbox`	transactional Event publication
`repositorysourceindex_idempotency`	duplicate-write protection
`repositorysourceindex_exception`	exception, claim, dispute, or hold
`repositorysourceindex_correction`	correction and supersession lineage
`repositorysourceindex_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Repository facts must be tied to a path, commit or runtime artifact.
- Conflicting catalogs and legacy paths remain visible until canonicalized.
- This atlas records contradictions; it does not modify implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter’s evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.